Curso Profissional Técnico de Gestão de Equipamentos Informáticos

Project PROTECT HOUSE

Relatório Final da Prova de Aptidão Profissional (PAP)

Discente: Marco Brito
Professor Orientador: Prof. Ricardo Pires
Diretor de Curso: Prof. Ricardo Pires
Diretora de Turma: Prof. Inês Silva

Sintra, Fevereiro 2026

AGRADECIMENTOS

A realização do projeto *Project PROTECT HOUSE* constituiu um desafio técnico e formativo exigente, cuja concretização só foi possível graças ao apoio de diversas pessoas e entidades.

Em primeiro lugar, expresso o meu sincero agradecimento aos professores do Curso Profissional de Técnico de Gestão de Equipamentos Informáticos da Escola Secundária Miguel Torga, pelo acompanhamento, orientação técnica e incentivo constante ao longo do desenvolvimento da Prova de Aptidão Profissional.

Agradeço igualmente aos meus colegas de curso, pelo espírito de entreajuda e partilha de conhecimentos, fundamentais para a superação de dificuldades técnicas e para o enriquecimento do processo de aprendizagem.

RESUMO

O presente relatório descreve o desenvolvimento do projeto *Project PROTECT HOUSE*, concebido no âmbito da Prova de Aptidão Profissional do Curso Profissional de Técnico de Gestão de Equipamentos Informáticos, ano letivo 2025/2026.

O projeto consiste na conceção e implementação de um sistema de alarme residencial baseado na plataforma Arduino Uno, integrando um sensor de movimento (PIR), um teclado matricial 4x4 para autenticação por código, um buzzer para sinalização sonora e um LED indicador de estado. O sistema foi estruturado segundo o modelo de máquina de estados, permitindo uma organização lógica e eficiente do seu funcionamento.

Ao longo do relatório são descritas as diferentes fases do projeto, desde a conceção inicial, passando pela arquitetura do sistema, desenvolvimento dos algoritmos e implementação do código, até à fase de testes e validação funcional.

O objetivo principal consistiu na aplicação prática de conhecimentos adquiridos ao longo do curso, nomeadamente nas áreas de eletrónica, programação e sistemas embarcados, promovendo simultaneamente competências de planeamento, organização e resolução de problemas técnicos.

ÍNDICE

1. Introdução 5

2. Cronograma 6
   2.1. Diagrama de Gantt 6

3. Corpo do Trabalho 7
   3.1. Conceção do Sistema 7
   3.2. Arquitetura do Sistema 7
      3.2.1. Bloco de Entrada 8
      3.2.2. Unidade de Processamento 8
      3.2.3. Bloco de Saída 9

   3.3. Máquina de Estados 9
      3.3.1. Descrição dos Estados 9
      3.3.2. Transições de Estado 10

   3.4. Algoritmo Geral do Sistema 10
   3.5. Algoritmo de Autenticação 11
   3.6. Implementação do Circuito Eletrónico 11
   3.7. Implementação da Sirene 12
   3.8. Testes e Validação 12
   3.9. Limitações Identificadas 13
   3.10. Propostas de Melhoria 13

4. Conclusão 14

5. Referências Bibliográficas e Sítios de Pesquisa 15

6. Anexos 16

1. INTRODUÇÃO

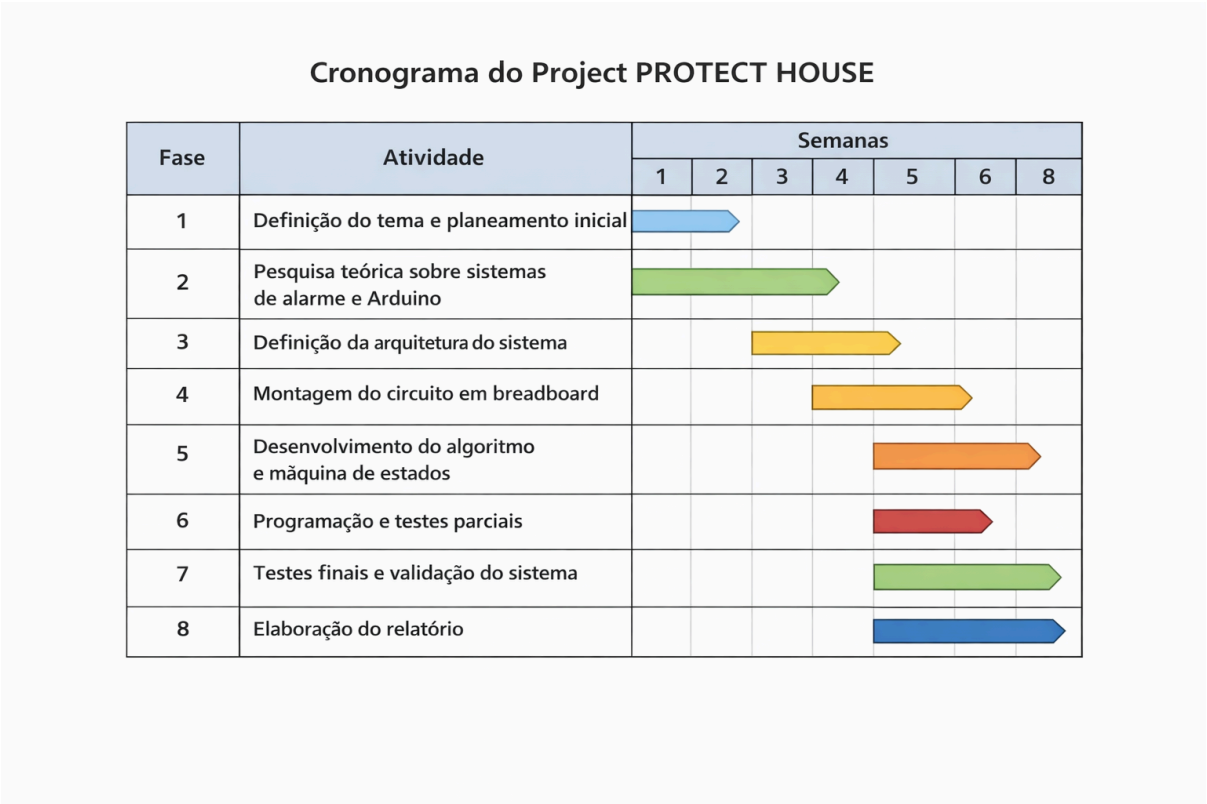
A crescente necessidade de segurança em ambientes residenciais tem conduzido ao desenvolvimento de soluções tecnológicas cada vez mais acessíveis e eficientes. Os sistemas de alarme eletrónicos representam uma das formas mais comuns de proteção contra intrusão, sendo amplamente utilizados tanto em habitações particulares como em instalações comerciais.

Neste contexto, surge o *Project PROTECT HOUSE*, um sistema de alarme residencial desenvolvido com base na plataforma Arduino Uno. Este projeto foi concebido no âmbito da Prova de Aptidão Profissional do Curso Profissional de Técnico de Gestão de Equipamentos Informáticos da Escola Secundária Miguel Torga, no ano letivo 2025/2026.

O principal objetivo deste projeto consistiu na criação de um sistema funcional capaz de:

- Detetar movimento através de um sensor PIR;
- Emitir um sinal sonoro de alarme em caso de intrusão;
- Permitir a ativação e desativação do sistema mediante introdução de um código de segurança;
- Organizar o funcionamento do sistema através de uma máquina de estados.Para além da componente técnica, este projeto permitiu consolidar competências adquiridas ao longo da formação, nomeadamente: Programação estruturada em linguagem C/C++;
- Implementação de sistemas baseados em microcontroladores;
- Interligação de componentes eletrónicos em breadboard;
- Aplicação de princípios de lógica sequencial e máquinas de estados;
- Planeamento e organização de um projeto técnico.

2. CRONOGRAMA



Cronograma do Project PROTECT HOUSE

Fase	Atividade	Semanas						
		1	2	3	4	5	6	8
1	Definição do tema e planeamento inicial							
2	Pesquisa teórica sobre sistemas de alarme e Arduino							
3	Definição da arquitetura do sistema							
4	Montagem do circuito em breadboard							
5	Desenvolvimento do algoritmo e máquina de estados							
6	Programação e testes parciais							
7	Testes finais e validação do sistema							
8	Elaboração do relatório							

O desenvolvimento do *Project PROTECT HOUSE* foi organizado de forma faseada, permitindo uma gestão eficiente do tempo e uma progressão estruturada das diferentes etapas do projeto. Para esse efeito, foi elaborado um planeamento temporal com base num modelo semelhante ao Diagrama de Gantt, no qual se distribuíram as atividades ao longo do período de realização da Prova de Aptidão Profissional.

A organização temporal do projeto teve como principais objetivos:

- Garantir o cumprimento dos prazos estabelecidos;
- Permitir a identificação antecipada de dificuldades técnicas;
- Assegurar uma evolução progressiva desde a conceção até à implementação final;
- Promover hábitos de organização e planeamento profissional.

3. CORPO DO TRABALHO

3.1. Conceção do Sistema

O *Project PROTECT HOUSE* foi concebido com o objetivo de desenvolver um sistema de alarme residencial funcional, fiável e estruturalmente organizado. A conceção do sistema iniciou-se com a definição dos requisitos funcionais, os quais determinaram a arquitetura, os componentes e a abordagem de programação adotada.

Foram estabelecidos os seguintes requisitos:

- O sistema deve iniciar-se em estado desarmado;
- Deve permitir ativação manual através de botão;
- Deve monitorizar continuamente a deteção de movimento;
- Deve emitir sinal sonoro contínuo em caso de intrusão;
- Deve permitir desativação apenas mediante introdução de código correto;
- Deve apresentar organização lógica clara através de máquina de estados.

A definição prévia destes requisitos permitiu evitar alterações estruturais durante a fase de implementação.

3.2. Arquitetura do Sistema

A arquitetura adotada segue um modelo modular dividido em três blocos fundamentais:

1. Bloco de Entrada
2. Unidade de Processamento
3. Bloco de Saída

Esta divisão assegura clareza estrutural e facilita eventuais melhorias futuras.

3.2.1. Bloco de Entrada

O bloco de entrada é composto por três elementos principais:

- Sensor PIR
- Teclado matricial 4x4
- Botão de ativação

O sensor PIR deteta variações de radiação infravermelha associadas à presença humana. Quando ocorre movimento, o sensor envia um sinal digital de nível lógico alto para o microcontrolador.

O teclado matricial permite a autenticação do utilizador através de código numérico. A utilização de uma matriz de 4 linhas e 4 colunas reduz o número de pinos necessários e permite identificação individual de cada tecla.

O botão físico permite iniciar o processo de ativação do sistema. Foi implementado controlo de anti-repique temporal para evitar leituras múltiplas não intencionais.

3.2.2. Unidade de Processamento

A unidade de processamento é assegurada pelo microcontrolador ATmega328P integrado no Arduino Uno.

Este microcontrolador executa o programa principal, responsável por:

- Ler entradas digitais;
- Avaliar condições lógicas;
- Gerir transições de estado;
- Controlar dispositivos de saída.

A programação foi realizada em linguagem C/C++.

3.2.3. Bloco de Saída

O bloco de saída é composto por:

- LED indicador de estado;
- Buzzer (sirene).

O LED fornece indicação visual do estado do sistema.
O buzzer emite sinal sonoro contínuo com variação de frequência quando o sistema entra em estado de disparo.

3.3. Máquina de Estados

A máquina de estados implementada constitui o núcleo lógico do sistema.

Foram definidos quatro estados distintos:

- STANDBY
- ARMING
- ARMED
- TRIGGERED

STANDBY

Estado inicial do sistema. O LED encontra-se desligado e a sirene inativa.

ARMING

Estado intermédio após ativação por botão. Permite pequena janela temporal antes de o sistema ficar totalmente armado.

ARMED

Estado de vigilância ativa. O sensor PIR é monitorizado continuamente.

TRIGGERED

Estado de alarme. A sirene é ativada e mantém-se em funcionamento até introdução do código correto.

A utilização deste modelo permite:

- Separação clara de responsabilidades;
- Facilidade de leitura e manutenção do código;
- Redução de erros lógicos;
- Escalabilidade futura.

3.4. Algoritmo Geral do Sistema

O algoritmo implementado segue uma estrutura sequencial baseada na máquina de estados.

Fluxo lógico:

1. Inicialização do sistema em STANDBY;
2. Monitorização contínua do botão;
3. Transição para ARMING após ativação;
4. Transição automática para ARMED após intervalo temporal;
5. Monitorização do sensor PIR;
6. Transição para TRIGGERED em caso de deteção;
7. Ativação da sirene;
8. Verificação contínua do teclado;
9. Retorno a STANDBY após código correto.

A lógica implementada garante previsibilidade e estabilidade operacional.

3.5. Algoritmo de Autenticação

O sistema de autenticação foi concebido com simplicidade e robustez.

Funcionamento:

- Apenas dígitos numéricos são aceites;
- A tecla "*" limpa o código introduzido;
- A tecla "#" confirma a introdução;
- O código digitado é comparado com o código armazenado internamente;
- Caso coincida, o sistema retorna ao estado STANDBY.

Foi definido um limite máximo de quatro dígitos para evitar armazenamento excessivo de dados na memória.

3.6. Implementação do Circuito Eletrónico

O circuito foi montado em breadboard, permitindo flexibilidade e facilidade de modificação.

Ligações principais:

- Sensor PIR ligado a pino digital do Arduino;
- Teclado matricial ligado a 8 pinos digitais;
- LED ligado a pino digital com resistência limitadora;
- Buzzer ligado a pino digital de saída;
- Botão ligado a pino digital com configuração adequada de entrada.

A alimentação do sistema foi realizada através da porta USB do Arduino, eliminando a necessidade de fonte externa.

A utilização de breadboard permitiu:

- Organização eficiente dos componentes;
- Facilidade de substituição;
- Segurança durante testes.

3.7. Implementação da Sirene

A sirene foi implementada utilizando variação periódica de frequência para simular um alarme real.

Foi utilizada abordagem não bloqueante com recurso à função `millis()`, evitando o uso de `delay()` prolongados que poderiam comprometer a leitura do teclado.

Esta decisão técnica permitiu:

- Manutenção da responsividade do sistema;
- Leitura contínua de entradas;
- Funcionamento estável da máquina de estados.

3.8. Testes e Validação

Após implementação do circuito e do código, foram realizados testes funcionais para validação do sistema.

Testes realizados:

- Verificação do estado inicial;
- Teste de ativação por botão;
- Teste de transição para estado armado;

- Teste de deteção de movimento;
- Teste de ativação da sirene;
- Teste de desativação por código correto;
- Teste de código incorreto.

Os resultados obtidos demonstraram que o sistema cumpre os requisitos inicialmente definidos.

3.9. Limitações Identificadas

Durante o desenvolvimento foram identificadas algumas limitações:

- Ausência de sistema de backup energético;
- Código fixo não configurável externamente;
- Ausência de comunicação remota;
- Sistema não preparado para múltiplas zonas de deteção.

Estas limitações constituem oportunidades de melhoria futura.

3.10. Propostas de Melhoria

Como evolução futura do projeto, poderiam ser implementadas:

- Integração com módulo GSM para envio de SMS;
- Implementação de display LCD;
- Armazenamento de código em EEPROM;
- Adição de múltiplos sensores;
- Alimentação por bateria com sistema de carregamento.

4. CONCLUSÃO

O desenvolvimento do *Project PROTECT HOUSE* constituiu um processo técnico e formativo exigente, permitindo a aplicação prática dos conhecimentos adquiridos ao longo do Curso Profissional de Técnico de Gestão de Equipamentos Informáticos.

O objetivo principal do projeto — conceber e implementar um sistema de alarme residencial funcional baseado na plataforma Arduino Uno — foi plenamente alcançado. O sistema desenvolvido demonstra capacidade de deteção de movimento através de sensor PIR, ativação de sinalização sonora contínua em caso de intrusão e desativação segura mediante autenticação por código numérico.

A utilização de uma máquina de estados revelou-se uma decisão estruturante e determinante para a organização lógica do sistema. Este modelo permitiu uma separação clara das responsabilidades de cada estado, garantindo previsibilidade no comportamento do sistema e facilitando a manutenção e evolução futura do código.

Do ponto de vista técnico, o projeto envolveu a integração coerente de componentes eletrónicos distintos, a implementação de algoritmos estruturados e a utilização de técnicas de programação não bloqueante, assegurando a responsividade contínua do sistema. A opção pela utilização da função millis() em detrimento de atrasos bloqueantes demonstrou maturidade técnica e compreensão do funcionamento interno do microcontrolador.

Durante o processo de desenvolvimento foram enfrentados diversos desafios, nomeadamente na fase de depuração do código, gestão de estados e estabilização da leitura do teclado. A superação destes obstáculos contribuiu significativamente para o desenvolvimento de competências de resolução de problemas, pensamento lógico e organização metodológica.

Para além dos resultados técnicos obtidos, este projeto representou um contributo relevante para a formação pessoal e profissional do aluno, promovendo autonomia, responsabilidade e rigor no cumprimento de prazos. A realização da PAP permitiu consolidar conhecimentos teóricos através da sua aplicação prática, reforçando a preparação para desafios futuros no domínio dos sistemas informáticos e eletrónicos.

Conclui-se, assim, que o *Project PROTECT HOUSE* cumpre os objetivos propostos, constituindo uma solução funcional, estruturada e tecnicamente fundamentada no âmbito da segurança eletrónica residencial.

5. REFERÊNCIAS BIBLIOGRÁFICAS E SÍTIOS DE PESQUISA

BANZI, Massimo; *Getting Started with Arduino*; Maker Media; 2011; Califórnia.

MONK, Simon; *Programming Arduino: Getting Started with Sketches*; McGraw-Hill Education; 2016; Nova Iorque.

PLATT, Charles; *Make: Electronics – Learning Through Discovery*; Maker Media; 2015; Estados Unidos da América.

MICROCHIP TECHNOLOGY; *ATmega328P Microcontroller Datasheet*; Microchip Technology Inc.; 2018; Estados Unidos da América.

Sítios de Pesquisa

AUTODESK. Tinkercad – Circuits.
https://www.tinkercad.com

ARDUINO. Arduino Official Documentation.
https://www.arduino.cc

AUTODESK. Tinkercad Learning Resources.
https://www.tinkercad.com/learn

ELECTRONICS TUTORIALS.
https://www.electronics-tutorials.ws

6. ANEXOS

anexo 1- circuito

anexo 2- código

Todas as alterações salvas

Código | Iniciar simulação | Enviar para

Texto | 1 (Arduino Uno R3)

```
#include <Keypad.h>

const bool DEBUG = true;
const long BAUD_RATE = 9600;

const byte PIN_LED    = 3;
const byte PIN_BUZZER = 4;
const byte PIN_PIR    = A5;
const byte PIN_BOTAO  = 2;

const String CODIGO_CORRECTO = "2745";

/* ===================== KEYPAD ===================== */
const byte LINHAS = 4;
const byte COLUNAS = 4;

char mapaTeclas[LINHAS][COLUNAS] = {
  {'1','2','3','A'},
  {'4','5','6','B'},
  {'7','8','9','C'},
  {'*','0','#','D'}
};

byte pinosLinhas[LINHAS]   = {13, 12, 11, 10};
byte pinosColunas[COLUNAS] = {9, 8, 7, 6};

Keypad keypad(makeKeymap(mapaTeclas), pinosLinhas, pinosColunas,

/* ===================== ESTADOS ===================== */
enum EstadoSistema : byte {
  STANDBY,
  ARMING,
  ARMED,
  TRIGGERED
};

EstadoSistema estadoAtual = STANDBY;

/* ===================== VARIÁVEIS ===================== */
String codigoDigitado = "";
```

Monitor serial

anexo 3- código

Código | Iniciar simulação | Enviar para

Texto | 1 (Arduino Uno R3)

```
38
39 /* ===================== VARIÁVEIS ===================== */
40 String codigoDigitado = "";
41
42 unsigned long tempoEstado = 0;
43 unsigned long tempoSirene = 0;
44
45 int freq = 700;
46 int passo = 5;
47
48 const int FREQ_MIN = 700;
49 const int FREQ_MAX = 900;
50
51 const unsigned long TEMPO_ARMING = 500;
52 const unsigned long INTERVALO_SIRENE = 10;
53
54 /* ===================== PROTÓTIPOS ===================== */
55 void logMsg(const char* msg);
56 void mudarEstado(EstadoSistema novoEstado);
57 bool processarTeclado(char tecla);
58 void atualizarSirene();
59
60 /* ===================== FUNÇÕES ===================== */
61 void logMsg(const char* msg) {
62   if (DEBUG) Serial.println(msg);
63 }
64
65 void mudarEstado(EstadoSistema novoEstado) {
66   estadoAtual = novoEstado;
67   tempoEstado = millis();
68
69   switch (estadoAtual) {
70     case STANDBY:
71       digitalWrite(PIN_LED, LOW);
72       noTone(PIN_BUZZER);
73       codigoDigitado = "";
74       logMsg("Estado: STANDBY");
75       break;
76
77     case ARMING:
78       codigoDigitado = "";
79
```

Monitor serial

anexo 4- código

Todas as alterações salvas

Código | Iniciar simulação | Enviar para

Texto | 1 (Arduino Uno R3)

```
      break;

    case ARMING:
      codigoDigitado = "";
      logMsg("Estado: ARMING");
      break;

    case ARMED:
      digitalWrite(PIN_LED, HIGH);
      codigoDigitado = "";
      logMsg("Estado: ARMED");
      break;

    case TRIGGERED:
      freq = FREQ_MIN;
      passo = 5;
      tempoSirene = millis();
      logMsg("Estado: TRIGGERED");
      break;
  }
}

bool processarTeclado(char tecla) {
  if (!tecla) return false;

  if (tecla == '*') {
    codigoDigitado = "";
    return false;
  }

  if (tecla == '#') {
    bool valido = (codigoDigitado == CODIGO_CORRECTO);
    codigoDigitado = "";
    return valido;
  }

  if (tecla >= '0' && tecla <= '9') {
    if (codigoDigitado.length() < 4) {
      codigoDigitado += tecla;
    }
  }
```

anexo 5- código

Todas as alterações salvas

Código | Iniciar simulação | Enviar para

Texto | 1 (Arduino Uno R3)

```
    return valido;
  }

  if (tecla >= '0' && tecla <= '9') {
    if (codigoDigitado.length() < 4) {
      codigoDigitado += tecla;
    }
  }

  return false;
}

void atualizarSirene() {
  if (millis() - tempoSirene < INTERVALO_SIRENE) return;
  tempoSirene = millis();

  tone(PIN_BUZZER, freq);

  freq += passo;
  if (freq >= FREQ_MAX || freq <= FREQ_MIN) {
    passo = -passo;
  }
}

/* ===================== SETUP ===================== */
void setup() {
  if (DEBUG) Serial.begin(BAUD_RATE);

  pinMode(PIN_LED, OUTPUT);
  pinMode(PIN_BUZZER, OUTPUT);
  pinMode(PIN_PIR, INPUT);
  pinMode(PIN_BOTAO, INPUT);   // pull-down externo

  digitalWrite(PIN_LED, LOW);
  noTone(PIN_BUZZER);

  mudarEstado(STANDBY);
}

/* ===================== LOOP ===================== */
void loop() {
```

Monitor serial

anexo 6- código

Todas as alterações salvas

Código | Iniciar simulação | Enviar para

Texto | 1 (Arduino Uno R3)

```
  digitalWrite(PIN_LED, LOW);
  noTone(PIN_BUZZER);

  mudarEstado(STANDBY);
}

/* ===================== LOOP ===================== */
void loop() {
  char tecla = keypad.getKey();

  if (estadoAtual != STANDBY && processarTeclado(tecla)) {
    logMsg("Código correcto. Sistema desarmado.");
    mudarEstado(STANDBY);
    return;
  }

  switch (estadoAtual) {

    case STANDBY:
      if (digitalRead(PIN_BOTAO) == HIGH) {
        mudarEstado(ARMING);
      }
      break;

    case ARMING:
      if (millis() - tempoEstado >= TEMPO_ARMING) {
        mudarEstado(ARMED);
      }
      break;

    case ARMED:
      if (digitalRead(PIN_PIR) == HIGH) {
        mudarEstado(TRIGGERED);
      }
      break;

    case TRIGGERED:
      atualizarSirene();
      break;
  }
}
```

Monitor serial